



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 4
Вариант 16
по курсу «Теория формальных языков»

Студент группы ИУ9-52Б Прадхан М. Р.

Преподаватель Непейвода А.Н.

Москва

15 января 2026

1 Задание

Дано расширенное регулярное выражение:

$$\wedge\left(\left((? : a|b)^*\right)\backslash 2\backslash 2 a\left((? : a|b)^*\right) b\backslash 3\right)\backslash 1?\$$$

Имеющуюся регулярку:

- Проанализировать на КС-свойство, в случае его наличия — на регулярность.
- Построить "наивный" парсер слов для языка, используя рекурсивный разбор с возвратами. Парсер не должен заикливаться.
- Построить оптимизированный парсер слов для языка. Оценить сверху его вычислительную сложность.
- Посредством фазз-тестирования проверить эквивалентность парсеров и построить сравнительные графики скорости их работы на случайных словах, принадлежащих языку, и не принадлежащих языку (два тестовых пула).

2 КС-свойство

Дано выражение:

$$\hat{\left(\left((? : a|b)^* \right) \setminus 2 \setminus 2 a \left((? : a|b)^* \right) b \setminus 3 \right) \setminus 1 ? \$}$$

Тогда обозначим:

$$x \in \{a, b\}^* \text{ — то, что захватывает } ((? : a|b)^*) \text{ (группа 2)}$$

$$y \in \{a, b\}^* \text{ — то, что захватывает } ((? : a|b)^*) \text{ (группа 3)}$$

Тогда внутренняя группа (группа 1) задаёт строку:

$$x x x a y b y$$

А всё выражение задаёт язык:

$$L = \{(xxxyby)^k \mid x, y \in \{a, b\}^*, k \in \{1, 2\}\}$$

Пересечём язык L с регулярным языком

$$R = a^* b a^* b a^* b a b$$

Тогда получаем:

$$L \cap R = \{a^n b a^n b a^n b a b \mid n \geq 0\}$$

С помощью леммы о накачке для контекстно-свободных языков докажем, что $L \cap R$, а значит и исходный язык, не является КС.

Пусть наше слово:

$$w = a^p b a^p b a^p b a b$$

Разобьём w на части $uvxyz$ так, чтобы выполнялись условия леммы о накачке:

$$|vxy| \leq p, \quad |vy| \geq 1, \quad uv^k xy^k z \in L \text{ для всех } k \geq 0$$

Рассмотрим все варианты накачки:

1. Накачивая отдельно блок из a^p , мы выходим из структуры языка.
2. Накачивая отдельно правые a или b , мы опять выходим из структуры языка.

3. Накачивая соседние блоки, состоящие из a^p , вне зависимости от синхронности, мы также выходим из языка.
4. Накачивая отдельно b в промежутках выходим из языка.

Следовательно, $L \cap R$ не контекстно-свободен, а значит и исходный язык L не является контекстно-свободным.

3 Наивный парсер

Наивный парсер следующим образом проверяет принадлежность слова языку, соответствующему регулярному выражению

$$\hat{\left(((? : a|b)^*) \setminus 2 \setminus 2 a ((? : a|b)^*) b \setminus 3 \right) \setminus 1 ? \$}$$

1. Перебираются все возможные длины префикса X слова.
2. Для каждого X проверяется условие обратной ссылки: оставшаяся часть слова либо пустая, либо равна X .
3. Внутри X перебираются все возможные длины Y , чтобы проверить, что префикс X имеет вид $YYYaZbZ$, где Z — произвольная подстрока.
4. Если найдена комбинация Y и Z , полностью удовлетворяющая структуре X , слово принимается; иначе — отклоняется.

4 Оптимизированный парсер

Оптимизированный парсер следующим образом проверяет принадлежность слова языку с регулярным выражением

$$\hat{\left(((? : a|b)^*) \setminus 2 \setminus 2 a ((? : a|b)^*) b \setminus 3 \right) \setminus 1 ? \$}$$

1. Перебираются все возможные длины Y , префикса подстроки X , который должен соответствовать $YYYaZbZ$.
2. Для каждого Y проверяется, что первые $3Y$ символа повторяют Y трижды (т.е. YYY).

3. Проверяется наличие символа **a** после YYY .
4. Перебираются все допустимые длины Z , и проверяется, что ZbZ полностью совпадает с соответствующими символами строки.
5. Вычисляется длина X как $3|Y| + 1 + 2|Z| + 1$. Проверяется остаток строки: если он пуст или равен X (для обратной ссылки `\1?`), слово принимается.

В отличие от наивного парсера, этот метод не перебирает все возможные X и минимизирует количество создаваемых строк `slice`, что ускоряет проверку на длинных словах.

Оценка вычислительной сложности

Обозначим n — длину слова.

- Перебор Y выполняется максимум $O(n)$ раз ($yLen \leq n/3$).
- Для каждого Y перебирается Z с длиной до $O(n)$.
- Внутри каждого Z выполняется проверка ZbZ и остатка X , которая требует максимум $O(n)$ операций посимвольно.

Таким образом, верхняя оценка сложности алгоритма:

$$O(n) \text{ (по } Y) \times O(n) \text{ (по } Z) \times O(n) \text{ (проверка } X/\text{остатка)} = O(n^3)$$

На практике реальная скорость гораздо выше из-за раннего выхода при несоответствиях и ограничений на длину Z , что сильно сокращает количество итераций.

5 Графики скорости работы парсеров

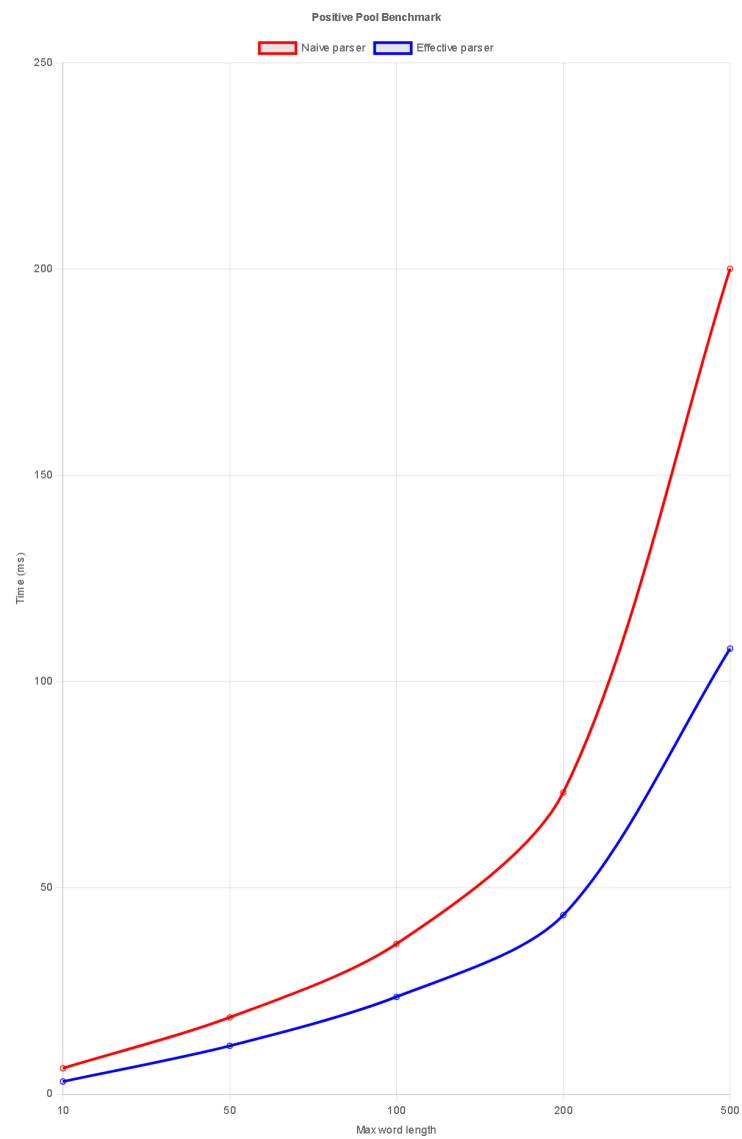


Рис. 1: График времени работы парсеров на пулле слов, принадлежащих языку

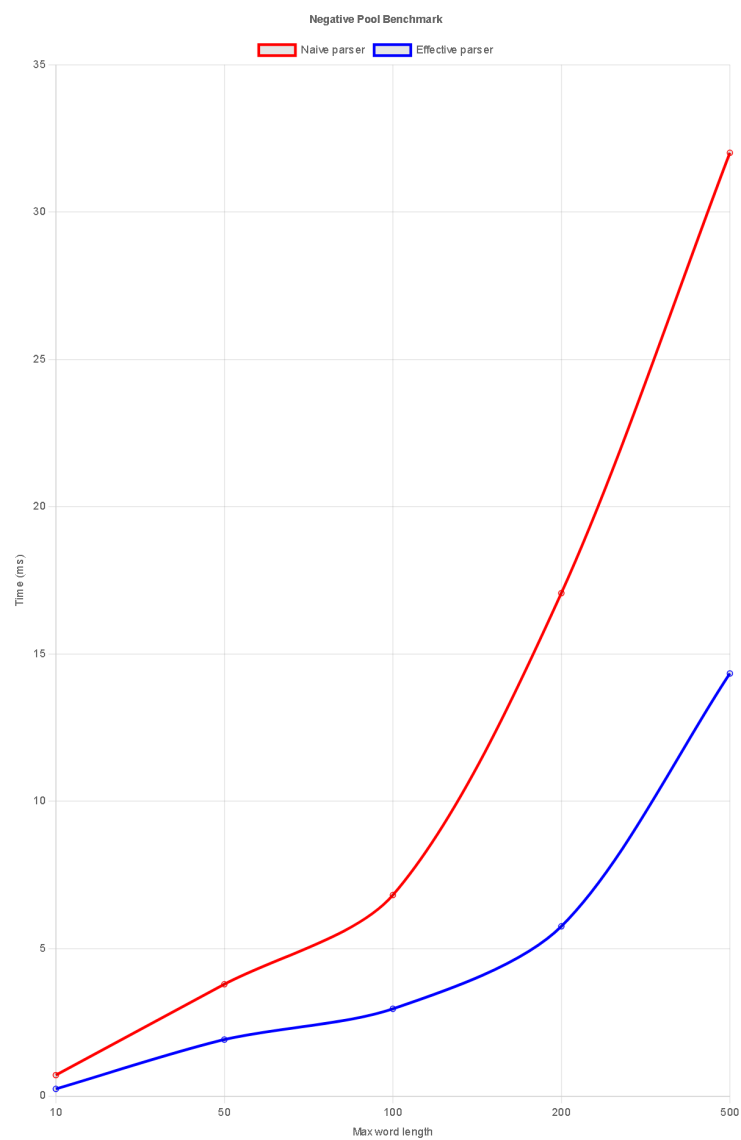


Рис. 2: График времени работы парсеров на пулле слов, не принадлежащих языку